



Woche 3: Trigonometrische Interpolation

Jonas Vonäsch

Numerical Methods for Computer Science

02.10.2026

Agenda

1. Clenshaw Recap
2. Trigonometrische Polynome
3. Fourier-Reihen
4. Diskrete Fourier-Transformation
5. Fourier in Numpy
6. Quiz review

Clenshaw Recap

Chebyshev-Interpolationspolynom

Natürlich gibt es auch eine Chebyshev-Interpolationsformel:

$$p(x) = c_0 + c_1 T_1(x) + \dots + c_n T_n(x)$$

Die Koeffizienten berechnen sich wie folgt:

$$c_0 = \frac{1}{n+1} \sum_{\ell=0}^n f(x_\ell)$$

Für $k = 1, \dots, n$:

$$c_k = \frac{2}{n+1} \sum_{\ell=0}^n f(x_\ell) T_k(x_\ell)$$

Chebyshev-Auswertung mit Clenshaw

Ziel: Auswertung von $p(x) = c_0 + c_1T_1(x) + \dots + c_nT_n(x)$

Rekursion (rückwärts):

$$d_{n+2} = d_{n+1} = 0$$

$$d_k = c_k + (2x)d_{k+1} - d_{k+2}$$

Auswertung:

$$p(x) = d_0 - xd_1 = \frac{1}{2}(d_0 - d_2)$$

Sehr stabil!!

Fazit



Trigonometrische Polynome

Trigonometrische Polynome

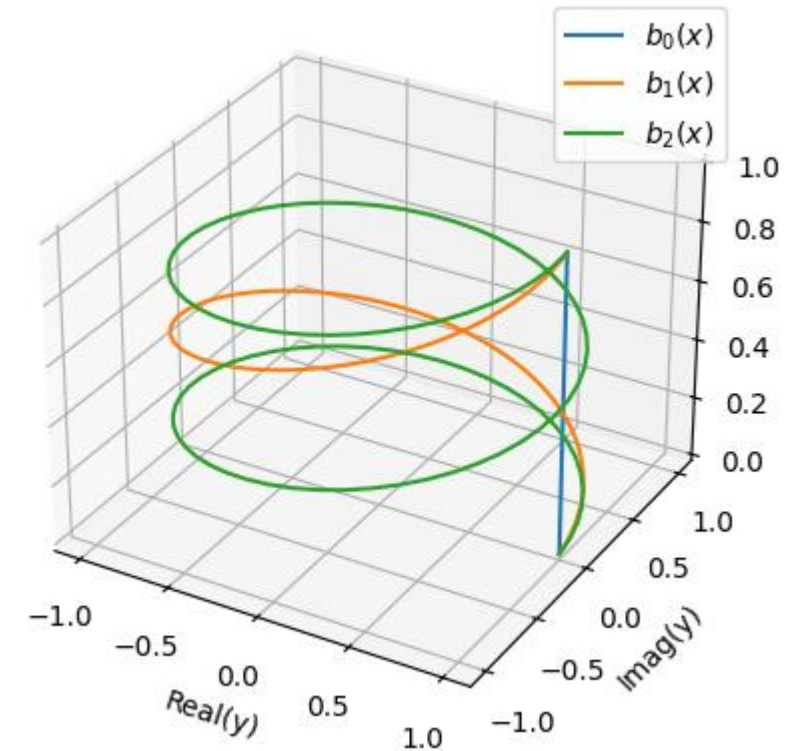
Definition:

$$p_m(t) := \sum_{k=-m}^m \gamma_k e^{2\pi i k t}$$

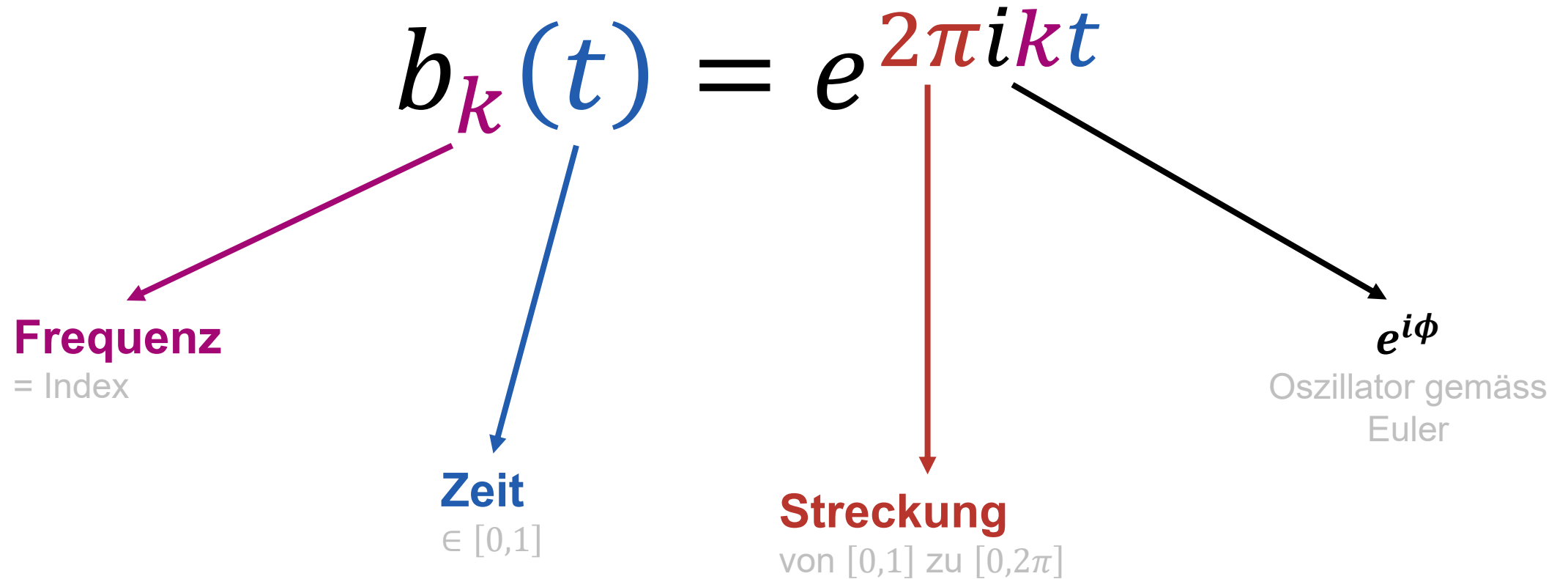
Basis:

$$b_k(t) := e^{2\pi i k t}$$

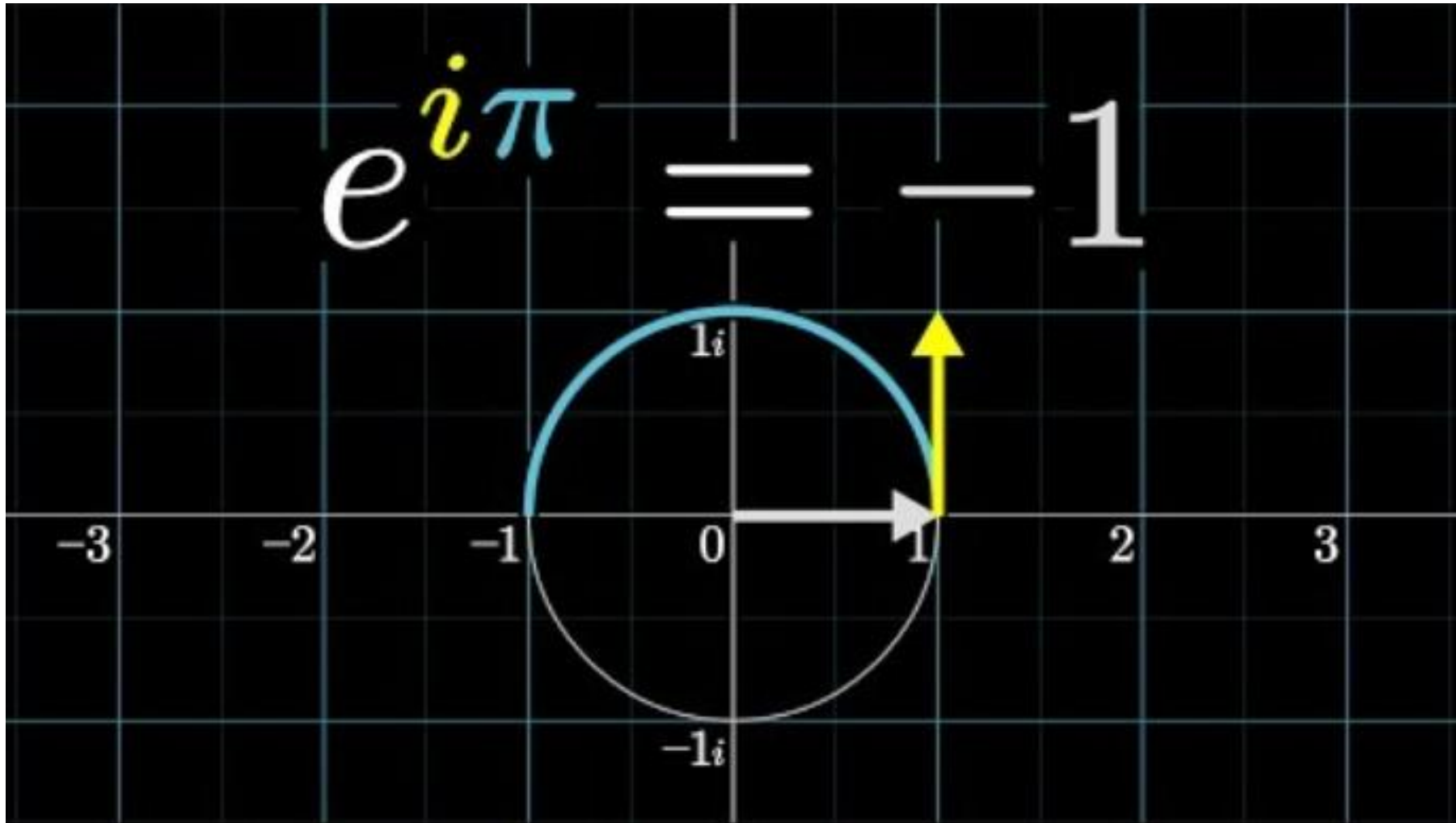
b_j sind **Orthogonal** $\Rightarrow$ Schnell konvertierbar



Anatomie der Basis-Oszillatoren

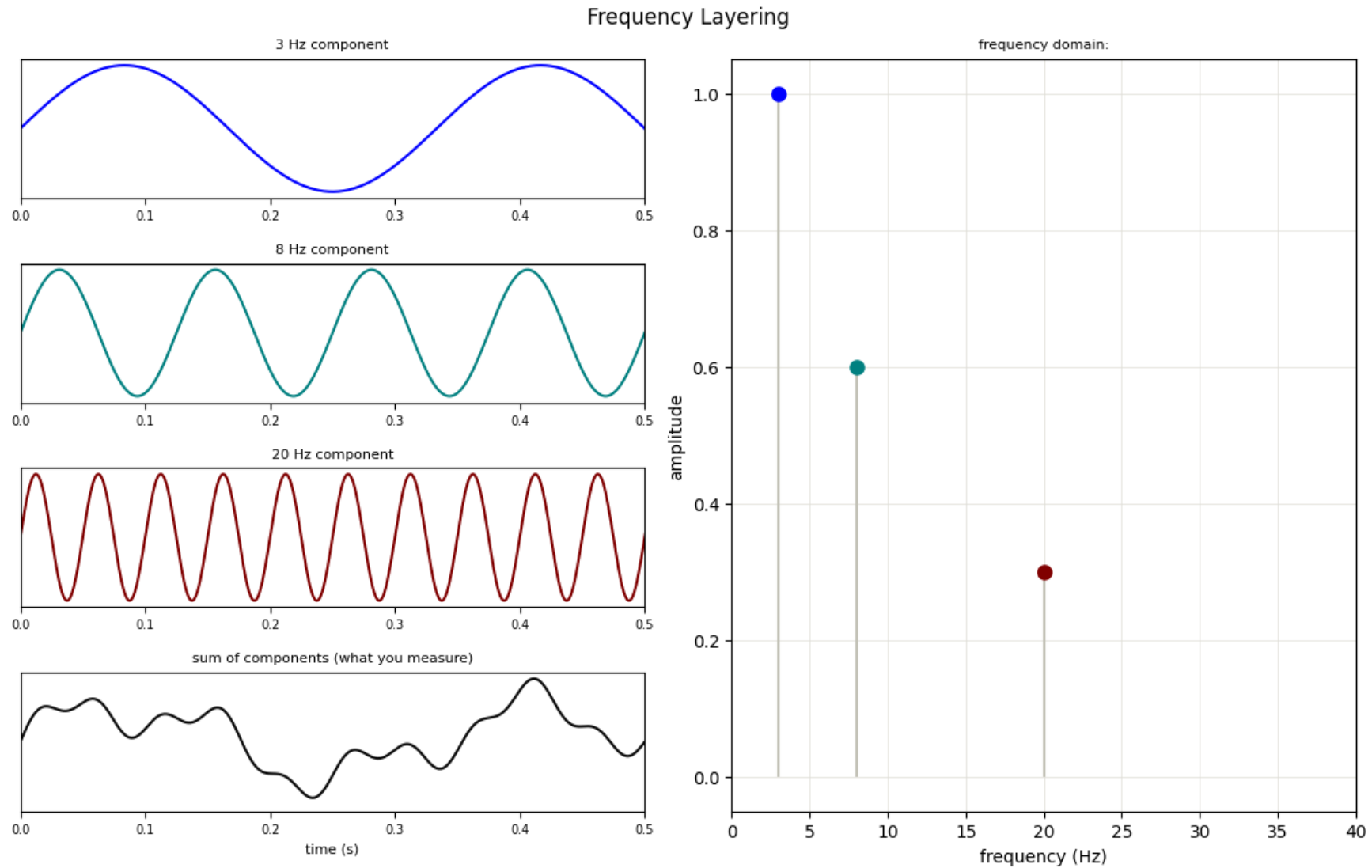


Anatomie der Basis-Oszillatoren: 3Blue1Brown



<https://youtu.be/v0YEaeICIKY>

Unser Ziel: Frequenzen Trennen



Reelle Schwingungen ~ Komplexe Schwingungen

Die Linearkombination:

$$a_j \cos(2\pi jt) + b_j \sin(2\pi jt)$$

Ist tatsächlich äquivalent zu

$$z_j e^{2\pi i j t} + \bar{z}_j e^{-2\pi i j t}$$

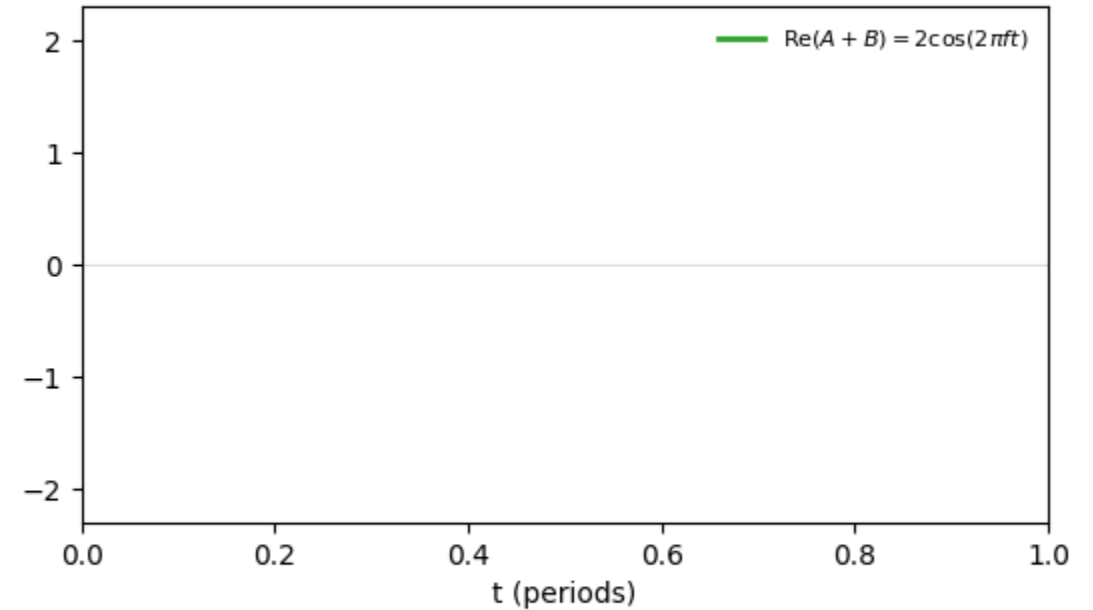
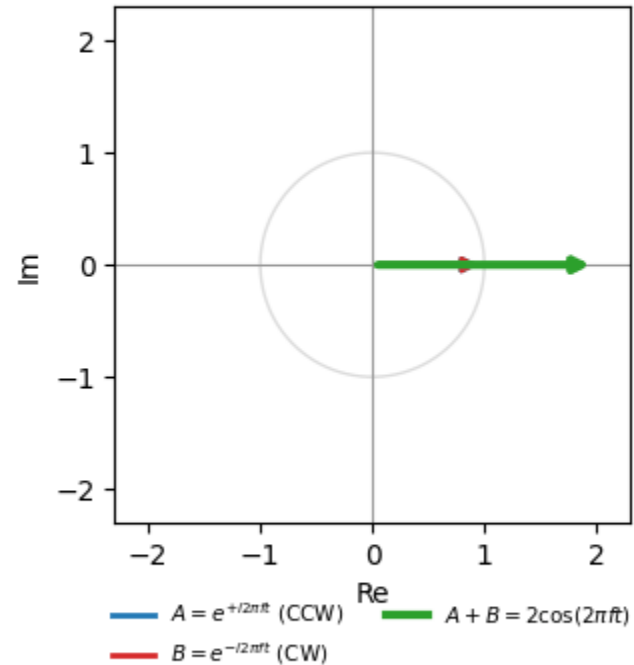
Für ein gewisses z_j

Dies lässt sich visuell zeigen...

Reelle Schwingungen ~ Komplexe Schwingungen

Why oscillation in $\mathbb{R}$ needs two oscillators:

conjugate phasor pair in the complex plane



Fourier-Reihen

Fourier-Reihen

Jede Funktion im Intervall $[0,1]$ lässt sich mit ihrer Fourier-Reihe annähern:

$$f(t) = \sum_{k=-\infty}^{\infty} \hat{f}(k) e^{2\pi i k t}$$

Wobei die **fourier-koeffizienten** $\hat{f}(k)$ wie folgt aussehen:

$$\hat{f}(k) = \int_0^1 f(t) e^{-2\pi i k t} dt$$

Die Funktion ist der L^2 -Grenzwert der Fourier-Reihe (Parseval'sche Gleichung):

$$\sum_{k=-\infty}^{\infty} |\hat{f}(k)|^2 = \|f\|_{L^2(0,1)}^2$$

Die Fkt. $\phi_k(t) = \exp(2\pi i k t)$ bilden eine **vollständige Orthonormalbasis** in $L^2(0,1)$.

Fourier-Reihen: **Achtung!!!**

Die Notation kann für Verwirrung Sorgen!

Es gelten folgende Analogien zur Newton- und Monom-Interpolation:

Fourier	Monom	Newton
$\hat{f}(k)$	a_k	β_k
$e^{2\pi i k t}$	t^k	$N_k(t)$

Ähnlicher zur bisherigen Notation:

$$\hat{f}_0 + \hat{f}_1 O_1(t) + \hat{f}_2 O_2(t) + \dots + \hat{f}_j O_j(t) + \dots$$

Mit Oszillatoren $O_j(t) = e^{2\pi i j t}$ und Koeffizienten $\hat{f}_j = \hat{f}(j)$. Nur für Intuition!

Anatomie einer Fourier-Reihe

$$f(t) = \sum_{k=-\infty}^{\infty} \hat{f}(k) e^{2\pi i k t}$$

The diagram illustrates the components of the Fourier series formula $f(t) = \sum_{k=-\infty}^{\infty} \hat{f}(k) e^{2\pi i k t}$. Three arrows point from parts of the formula to explanatory text:

- A green arrow points from the summation index $k = -\infty$ to $\pm$ Schwingungen.
- A red arrow points from the coefficient $\hat{f}(k)$ to Koeffizient, with the subtitle Amplitude der Schwingung.
- A blue arrow points from the exponential term $e^{2\pi i k t}$ to Basisvektor, with the subtitle Frequenz der Schwingung.

Reelle Fourier-Reihen

Für reellwertige Funktionen können wir dies auch darstellen als:

$$p_m(t) = \frac{a_0}{2} + \sum_{j=1}^{\infty} (a_j \cos(2\pi jt) + b_j \sin(2\pi jt))$$

Reellwertiges trig. Polynom mit Basis:

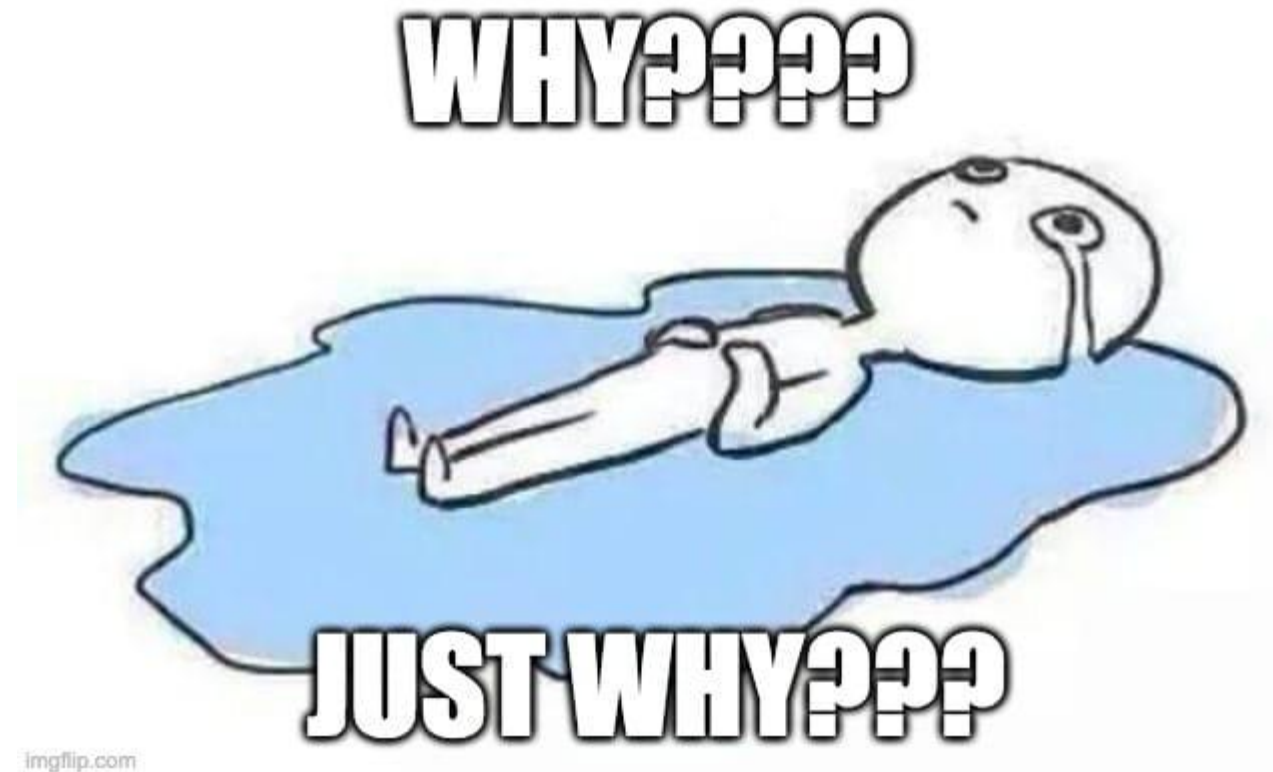
$$\begin{array}{c} \cos(2\pi jt) \\ \sin(2\pi jt) \end{array}$$

Reelle Fourier-Reihe Koeffizienten:

$$\begin{aligned} a_k &= 2 \int_0^1 f(t) \sin(2\pi kt) dt \\ b_k &= 2 \int_0^1 f(t) \cos(2\pi kt) dt \end{aligned}$$

WHY!?

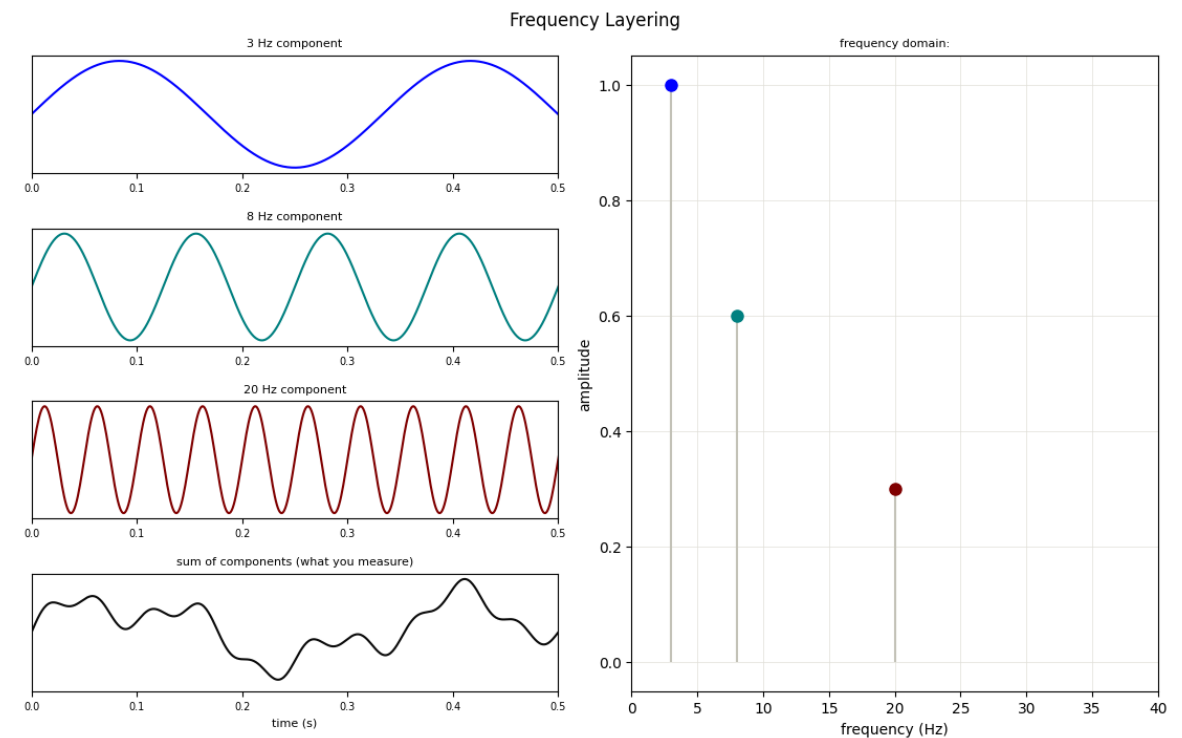
Wieso Fourier-Reihen?



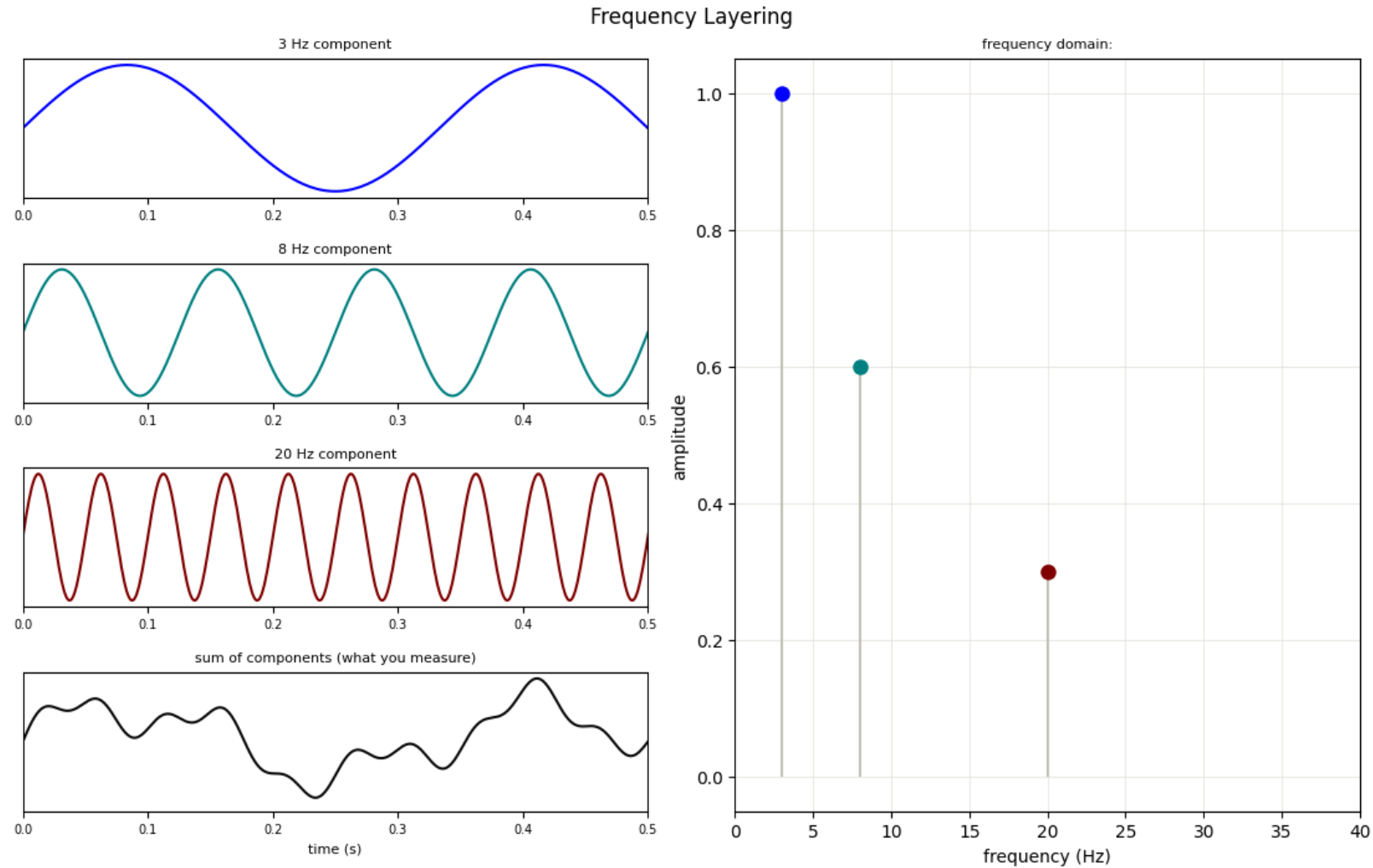
Wieso Fourier-Reihen?

Wieso Furier?

- Konvergiert schnell, wenn signal glatt ist
- Nähert *Jedes* Signal an
- Erlaubt Darstellung in der 'Frequency Domain'
- Kann Signale filtern

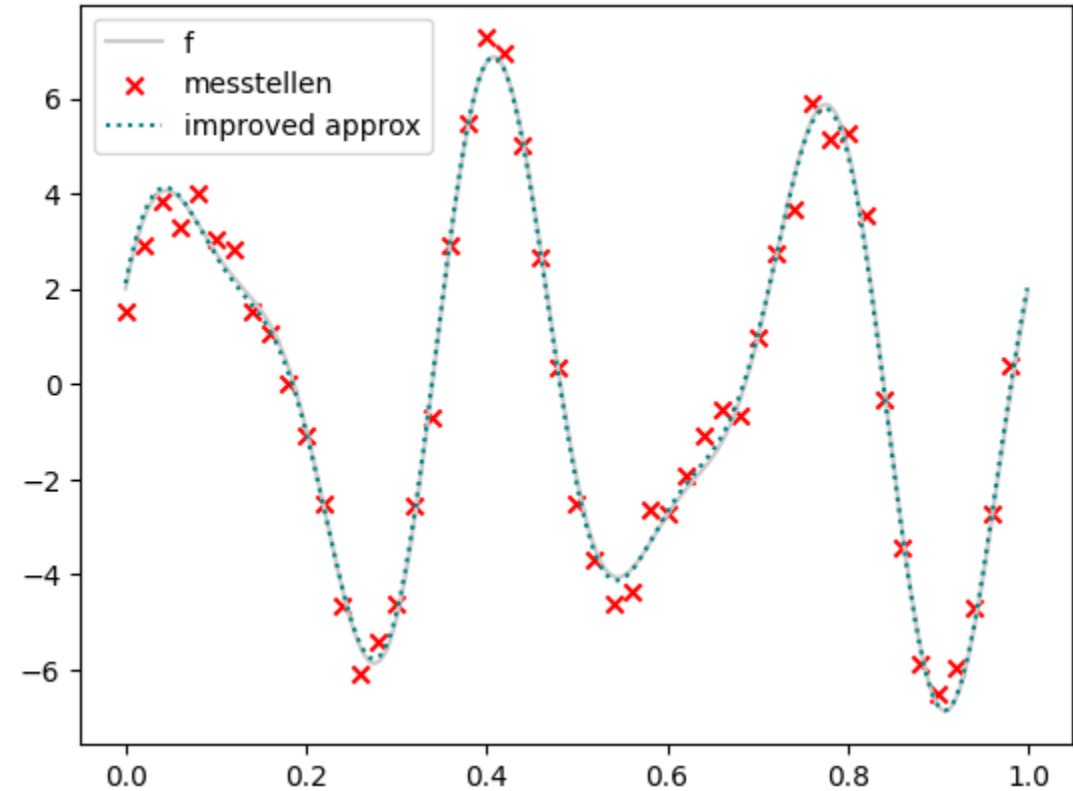
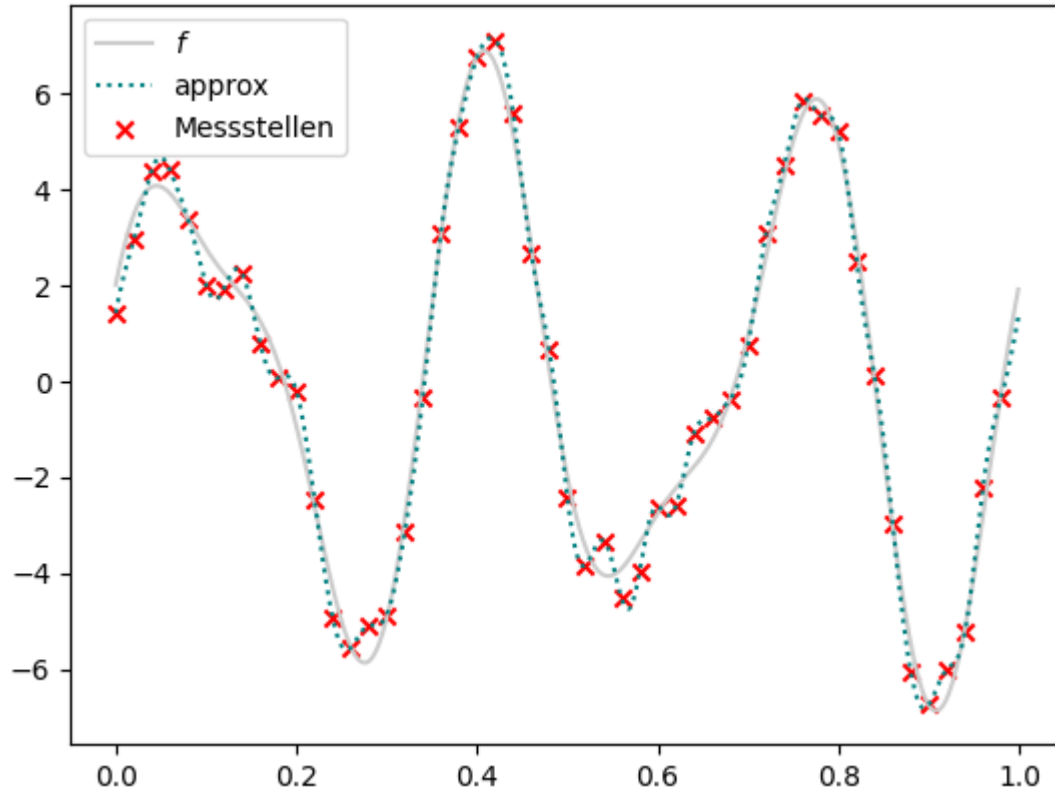


“Frequency Domain”



Fourier-Transformation: Filterung

Kann in der Frequency Domain die schwachen Frequenzen entfernen.



Diskrete Fourier-Transformation

Fourier-Reihen: Annäherung

Die analytischen Formeln lassen sich oft nicht direkt berechnen. Deswegen werden sie angenähert mit N Fourier-Koeffizienten an N Stützstellen $t_\ell = \frac{\ell}{N}$:

$$f(t) = \sum_{k=-\infty}^{\infty} \hat{f}(k) e^{2\pi i k t} \approx \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} \hat{f}(k) e^{2\pi i k t} =: p_{N-1}(t)$$

Summe
abschneiden

$$\hat{f}(k) = \int_0^1 f(t) e^{-2\pi i k t} dt \approx \frac{1}{N} \sum_{\ell=0}^{N-1} f(t_\ell) e^{-2\pi i k t_\ell} =: \hat{f}_N(k)$$

Riemann-
Annäherung

Und kombiniert:

$$\tilde{p}_{N-1}(t) := \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} \hat{f}_N(k) e^{2\pi i k t}$$

Wir sind fast so weit!

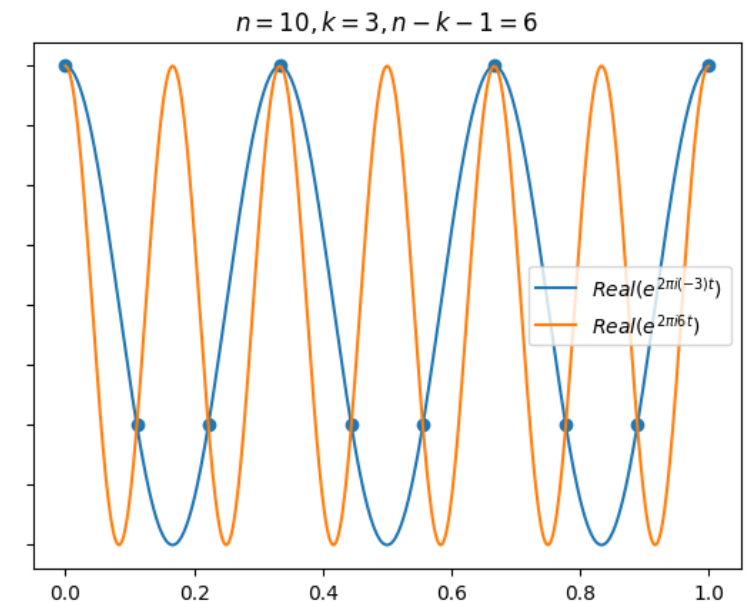
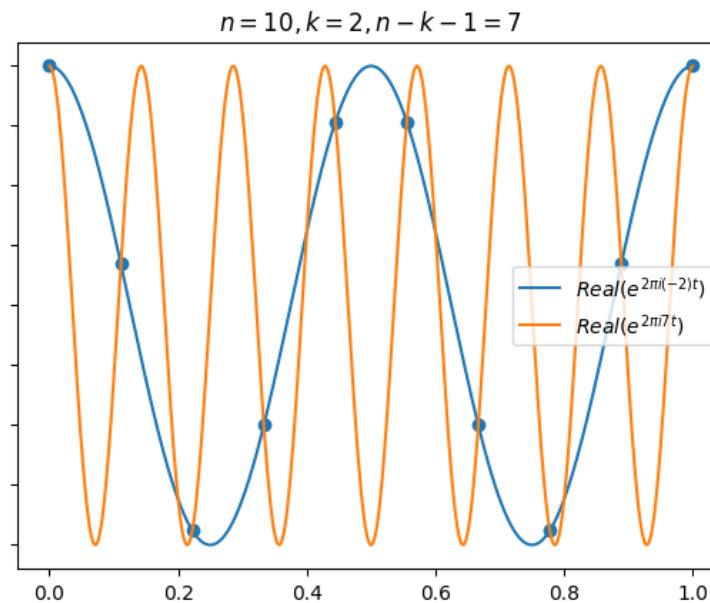
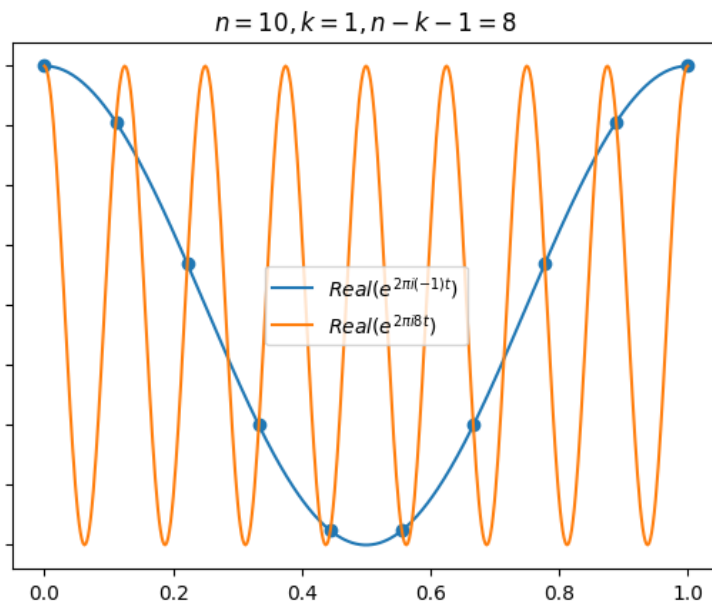
$$\tilde{p}_{N-1}(t_j) := \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} \hat{f}_N(k) e^{2\pi i k t_j} = \sum_{k=0}^{N-1} \hat{f}_N(k) e^{2\pi i k t_j}$$

Wait What???

Negative Frequenzen Analog zu Hohen Frequenzen?

Bei N Messstellen $T = [t_0 \quad t_1 \quad \dots * t_{(N-1)}]$ gilt:

$$e^{2\pi i(-k)t_j} = e^{2\pi i(N-k-1)t_j}$$



DIE FORMEL

$$\tilde{p}_{N-1}(t_j) = \sum_{k=0}^{N-1} \hat{f}_N(k) e^{2\pi i k t_j}$$

Jetzt noch effizient die $\hat{f}_N(k)$ finden mit der...

Fourier-Matrix

DFT: Ansatz

Seien $\omega_N = \exp\left(-\frac{2\pi i}{N}\right)$ die **Einheitswurzeln** und $w_N = \overline{\omega_N}$.

ω_N^{kt} ist ein Oszillator

Die folgenden Vektoren sind die *trigonometrische Basis* von $\mathbb{C}^N$ (Orthogonal):

$$v_0 = \begin{bmatrix} w_N^{0 \cdot 0} \\ w_N^{1 \cdot 0} \\ \vdots \\ w_N^{(N-1) \cdot 0} \end{bmatrix}, \dots, v_k = \begin{bmatrix} w_N^{0 \cdot k} \\ w_N^{1 \cdot k} \\ \vdots \\ w_N^{(N-1) \cdot k} \end{bmatrix}, \dots, v_{N-1} = \begin{bmatrix} w_N^{0 \cdot (N-1)} \\ w_N^{1 \cdot (N-1)} \\ \vdots \\ w_N^{(N-1) \cdot (N-1)} \end{bmatrix}$$

Heisst für die Matrix $V = [v_0 \ \dots \ v_{N-1}]$:

$$V^H V = N * I_N$$

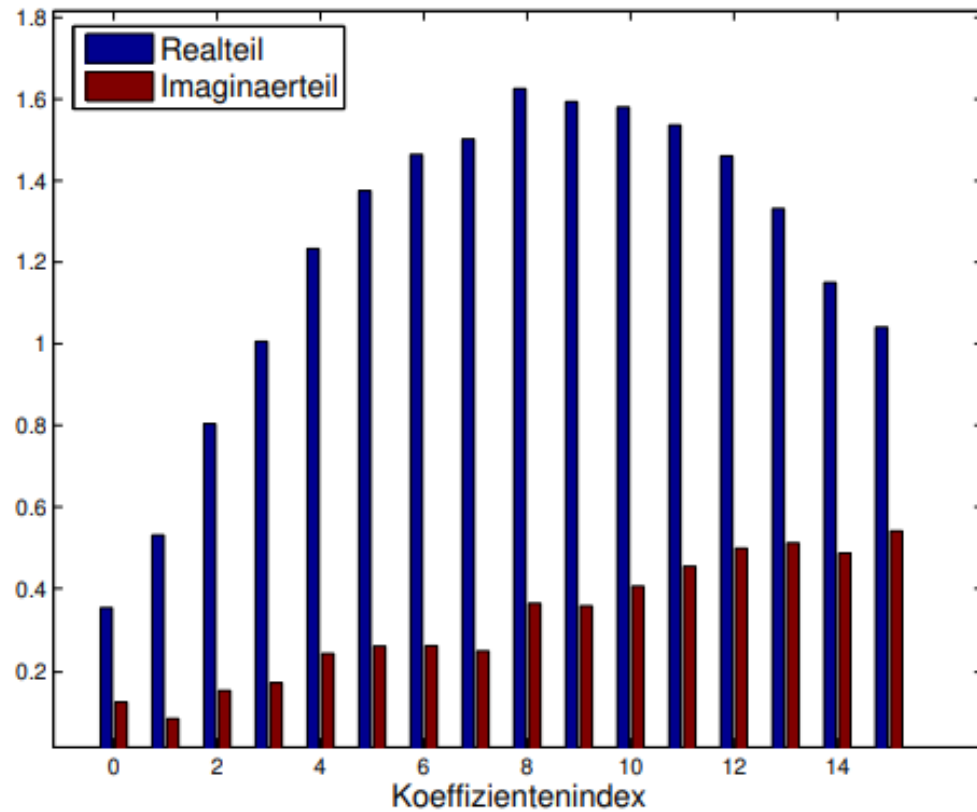
Fourier-Matrix

$$\mathbf{F}_N = \mathbf{V}^H = \begin{bmatrix} \omega_N^0 & \omega_N^0 & \dots & \omega_N^0 \\ \omega_N^0 & \omega_N^1 & \dots & \omega_N^{N-1} \\ \omega_N^0 & \omega_N^2 & \dots & \omega_N^{2(N-1)} \\ \vdots & \vdots & & \vdots \\ \omega_N^0 & \omega_N^{N-1} & \dots & \omega_N^{(N-1)^2} \end{bmatrix} = \left(\omega_N^{jk} \right)_{j,k=0}^{N-1} \in \mathbb{C}^{N \times N}$$

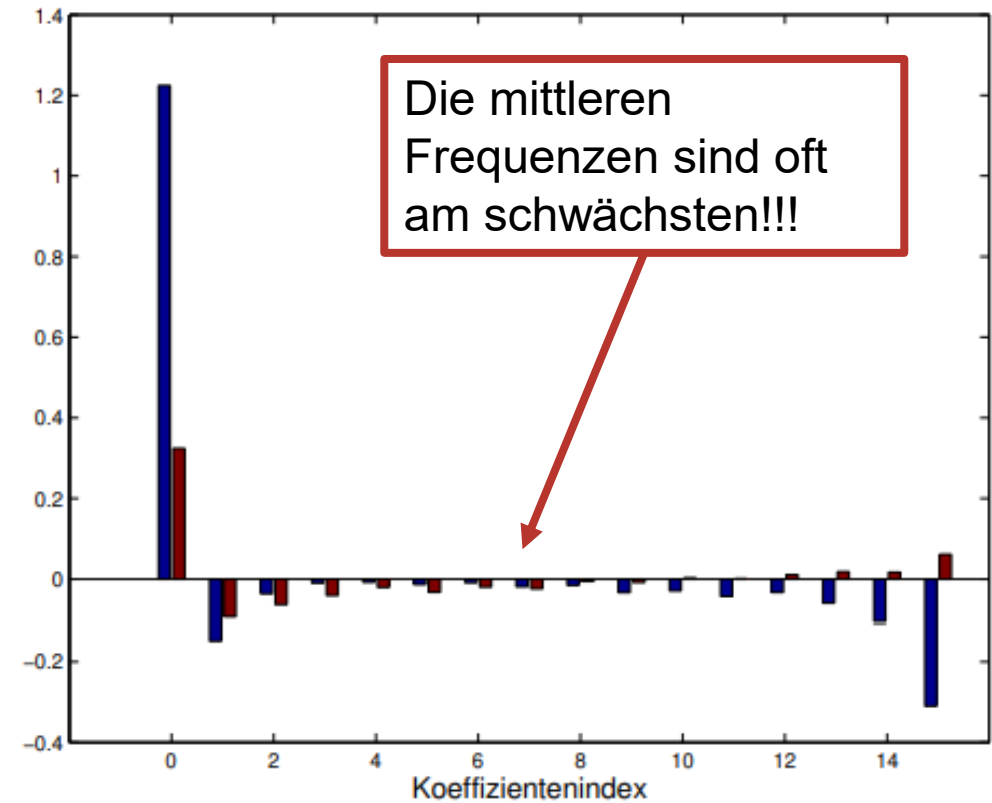
Dank Orthogonalität:

Multiplikation mit F_N entspricht der Diskreten Fourier-Transformation!

Diskrete Fourier-Transformation



Vektor in der Standardbasis



Vektor in der trigonometrischen Basis

Fourier in Numpy

Fourier in Numpy

`np.fft.fft:` Nimmt Messwerte $[x_0 \ x_1 \ \dots \ x_{N-1}]$ an Stellen $\left[\frac{0}{N} \ \frac{1}{N} \ \dots \ \frac{N-1}{N}\right]$ entgegen,
Gibt Fourier-Koeffizienten $[\hat{f}_N(0) \ \hat{f}_N(1) \ \dots \ \hat{f}_N(N-1)]$ zurück.

`np.fft.ifft:` Nimmt Fourier-Koeffizienten $[\hat{f}_N(0) \ \hat{f}_N(1) \ \dots \ \hat{f}_N(N-1)]$ entgegen,
Gibt Auswertung $[x_0 \ x_1 \ \dots \ x_{N-1}]$ an Stellen $\left[\frac{0}{N} \ \frac{1}{N} \ \dots \ \frac{N-1}{N}\right]$ zurück.

`np.fft.fftshift:` Nimmt Fourier-Koeffizienten $[\hat{f}_N(0) \ \hat{f}_N(1) \ \dots \ \hat{f}_N(N-1)]$ entgegen,
Gibt diese umgeordnet zurück: $\left[\hat{f}_N\left(-\frac{N}{2}\right) \ \hat{f}_N\left(-\frac{N}{2} + 1\right) \ \dots \ \hat{f}_N\left(\frac{N}{2} - 1\right)\right]$

Coding Session...



Jonas Vonäsch
Teaching Assistant NumCS
jvonaesch@student.ethz.ch

jvonaesch.github.io